JAVA 17 / MAVEN / JUNIT 5

Visitor Pattern
Advanced File System

Student: Zhassyn Zhalynuly **Group:** SE-2527

Repository: github.com/zzhassyn/sdp-assignment-6

1. Purpose and object model

The program models a small file system in memory. It supports three independent operations: total file size, filtered file search, and tree statistics. It follows the assignment requirements [1] and the Visitor structure shown in Lecture 9 [2]. It does not access real files.

Element defines accept(Visitor). The abstract **Node** class stores a name and parent reference. **FileNode** adds a non-negative byte size; **DirectoryNode** owns child nodes. Names and file sizes are immutable. The UML appendix shows the interfaces and all three visitors.

2. Double dispatch

First, root.accept(this) reaches the concrete element's method. Second, that method calls visitor.visit(this). Inside FileNode, the argument has type FileNode, so Java selects the file overload; inside DirectoryNode, it selects the directory overload. The concrete visitor supplies the operation. No runtime type-checking chain, reflection or type-selection cast is needed.

Visitor	Operation
TotalSizeVisitor	Adds each visited file size exactly once; directories add no bytes.
SearchVisitor	Requires a case-sensitive filename substring and size >= minimumSize.
StatisticsVisitor	Counts files and directories and records maximum depth.

3. Traversal, paths and state

Visitors own traversal. Every directory visit loops through its children and calls accept once per child. Element accept methods only dispatch. Each complete run starts with calculate(root), search(root) or collect(root), which resets accumulated state. Visitors can be reused sequentially and do not change the tree.

Search builds paths during traversal, saves and restores the parent path, omits the chosen directory root, uses /, and sorts results lexicographically. A file selected as the root uses its own filename. Depth is the number of edges from the selected root: root depth is **0**. Statistics include the root directory and empty directories.

4. Tree integrity

Before adding a child, DirectoryNode rejects duplicate sibling names, an existing parent, self-cycles and ancestor-cycles. It follows parent links to detect cycles. Invalid changes leave the tree unchanged. Names cannot be blank, dot segments or contain path separators; file sizes cannot be negative. Child lists and search results are immutable snapshots.

Demonstration and Clean Code

5. Original example data

The selected root is **study**. Its directories are docs, docs/archive, src and empty. Together with study, this gives five directories. The sizes below are sample metadata created in Main.

File path relative to study	Bytes
readme.txt	120
docs/plan.txt	200
docs/notes.txt	450
docs/archive/old-notes.txt	80
src/Main.java	900
src/Visitor.java	300

Run from the folder containing pom.xml: mvn -q compile exec:java. The following output was captured from the program.

```
Assignment 6 - Visitor Pattern
Student: Zhassyn Zhalynuly
Group: SE-2527

Total size: 2050 bytes

Search: name contains ".txt", size >= 200 bytes
docs/notes.txt
docs/plan.txt

Statistics:
Files: 6
Directories: 5
Maximum depth: 3
```

plan.txt is included because 200 equals the minimum size. The total is 2,050 bytes. The deepest file is docs/archive/old-notes.txt at depth 3.

6. Five Clean Code practices

Practice and code reference	Justification
Meaningful naming minimumSize; getMaxDepth()	Names describe what values mean without needing code comments.
Single Responsibility DirectoryNode; each visitor	Tree rules and independent operations live in separate classes.
Small methods accept(); joinPath(); checkDuplicateName()	Each method handles one short, readable step.
Encapsulation Node fields; getChildren()	Private data and immutable lists prevent callers from bypassing tree rules.
No type-checking chains FileNode.accept; DirectoryNode.accept	Overloaded visit methods choose behavior through dispatch.

Verification and extension

7. Automated tests

Run mvn test from the project root. The suite was executed on **8 November 2026** using Eclipse Temurin **17.0.21**, Maven **3.9.16**, JUnit **5.11.4** and Surefire **3.5.2**. Compilation targets Java 17. Test fixtures are independent of the demonstration.

Test class	Tests	Result
ElementTest	11	Passed
TotalSizeVisitorTest	5	Passed
SearchVisitorTest	8	Passed
StatisticsVisitorTest	7	Passed

Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Checkpoint	Verification
T1 / T2	Both dispatch overloads; totals across several nesting levels.
T3 / T4	Both filters, exact threshold, sorted paths and chosen subtree roots.
T5 / T6	Counts, depth, deeper empty directories, empty roots and no matches.
T7 / T8	Repeated runs and operation order; invalid mutations without corruption.

Extra checks cover immutable snapshots, zero-byte files, sizes above the integer range, and recovery after long overflow. Recorded results are in docs/test-results.txt.

8. Extension boundary and trade-offs

Commit **98e03f8** implements the tree, size/search visitors and 24 tests. Commit **6fbbaaf** adds StatisticsVisitor, seven tests and its use in Main. Element, Node, FileNode, DirectoryNode and Visitor remained byte-for-byte unchanged. The new visitor uses the original two visit methods; neither concrete accept method changed.

A new operation can be added as another visitor. A **new element type** requires a new visit overload in Visitor and an implementation in every existing visitor. This is the main trade-off: Visitor suits a stable set of element types. Visitor-owned recursion is simple, but repeats the child loop. A shared walker could remove that repetition at the cost of an extra path/depth contract.

The implementation supports sequential operations on an unchanged tree. Very deep recursion is limited by the Java call stack. Total-size overflow is rejected with ArithmeticException. These limits keep the example small and predictable.

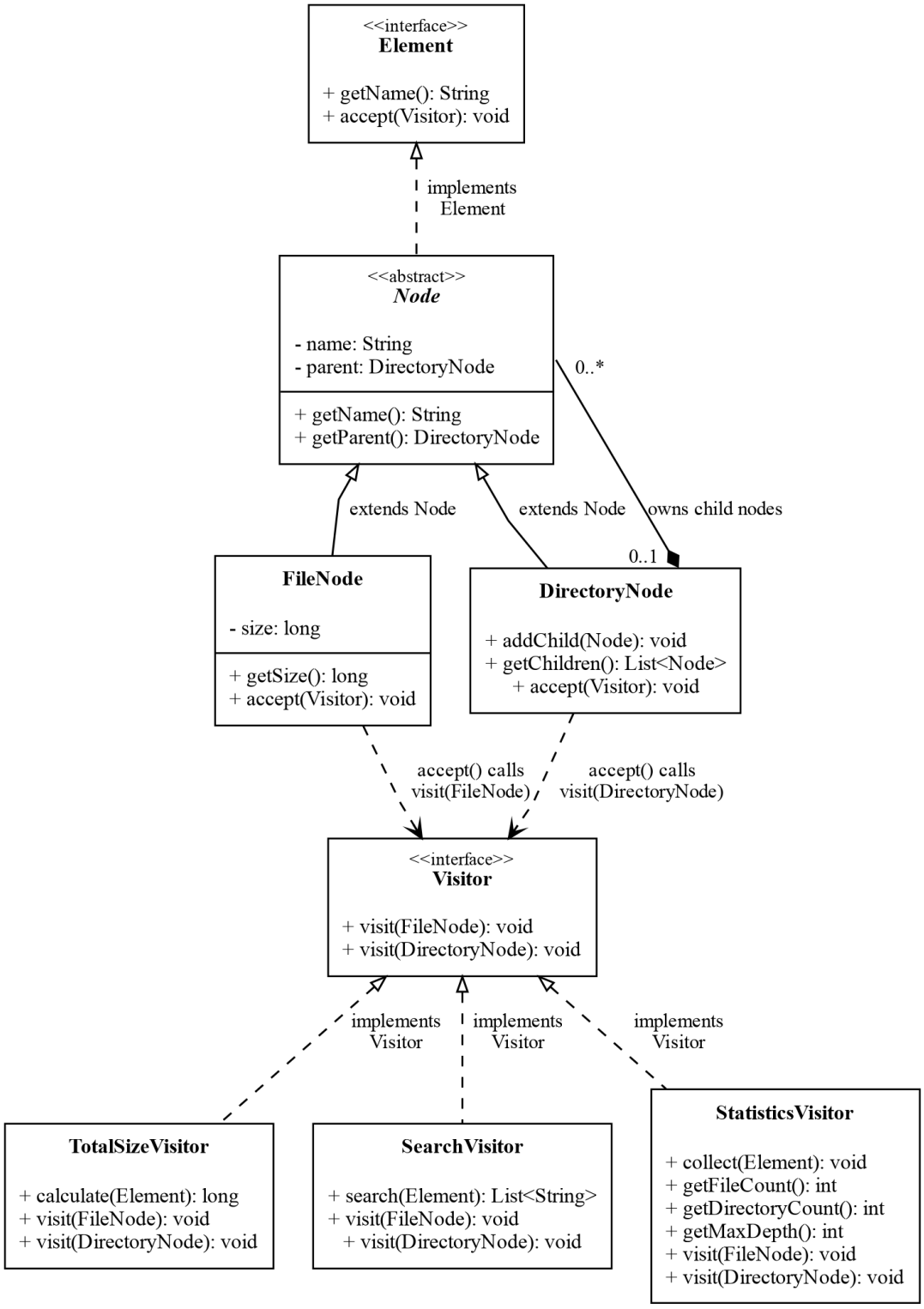
References

[1] Assignment_6_Visitor_Pattern.pdf, course handout, pp. 1-2.
[2] Lecture 9.pdf, Visitor structure and dispatch, pp. 10-11, 18 and 31.
[3] JUnit 5.11.4 User Guide - test framework and Maven setup.
[4] Maven Compiler Plugin 3.13.0: setting the Java release.

Repository: https://github.com/zzhassyn/sdp-assignment-6

Appendix - UML class diagram

All nine relationships have English labels. Arrowheads identify the target class or interface; the diamond marks ownership. Main is omitted for clarity.



Read the labels: extends Node means inheritance. **implements Element / Visitor** means interface implementation. **accept() calls visit(...)** shows dispatch to the matching visitor method. The filled diamond marks the owning directory: **0..*** means zero or more children, and **0..1** means zero or one parent for each node.